

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline for Customer Order Processing

-

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregates for business intelligence purposes.

Scope

- In Scope:
 - Ingestion of customer and order CSV files from specified S3 locations
 - Data cleaning operations (null removal, deduplication)
 - Registration of datasets in AWS Glue Data Catalog
 - SCD Type 2 implementation for order summary using Apache Hudi
 - Incremental processing logic triggered by customer updates
 - Generation of customer aggregate spend analytics table
 - Storage of curated and analytics datasets in S3
- Out of Scope:
 - Real-time streaming ingestion
 - Data archival and retention policies
 - User interface or reporting layer implementation
 - Authentication and authorization mechanisms
 - Cost optimization strategies

Assumptions

1. CSV files in source S3 locations follow consistent formatting and encoding (UTF-8)
2. CustId serves as the primary key for customer data and foreign key in order data
3. OrderId serves as the primary key for order data
4. "Null" (string) and NULL (actual null) are the only null representations requiring removal
5. Duplicate detection is based on all columns in each dataset

6. Date fields are in a parseable format (specific format not provided in BRD)
7. SCD2 change detection is based on any change in customer or order attributes
8. AWS Glue job has appropriate IAM permissions for S3, Glue Catalog, and CloudWatch
9. Hudi library is available in the Glue job environment
10. The pipeline runs on a scheduled or triggered basis (frequency not specified)
11. TotalAmount calculation is $\text{PricePerUnit} \times \text{Qty}$ aggregated per customer
12. OpTs (Operation Timestamp) represents the time of record processing
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a structured format for processing.

- Business Objective:

Enable downstream processing by making customer master data available in a queryable format.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (datatype not specified)
 - Name: (datatype not specified)
 - EmailId: (datatype not specified)
 - Region: (datatype not specified)
- Processing Rules / Functional Steps:
 1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files present in the location
 3. Parse CSV content with header row detection
 4. Load data into a DataFrame structure preserving the schema: CustId, Name, EmailId, Region
- Outputs:
 - In-memory customer dataset ready for data quality processing

- Preconditions:
- S3 bucket and path exist and are accessible
- CSV files contain header row matching the expected schema
- AWS Glue job has read permissions on the S3 location
- Postconditions:
- Customer data is loaded into memory
- Record count is logged for audit purposes
- Error / Validation Conditions:
- If S3 path does not exist, log error and halt processing
- If CSV files are malformed or unreadable, log error and halt processing
- If schema does not match expected columns, log warning and attempt best-effort mapping
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a structured format for processing.

- Business Objective:

Enable order transaction analysis by making transactional data available for joining with customer data.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified)
- ItemName: (datatype not specified)
- PricePerUnit: (datatype not specified)
- Qty: (datatype not specified)
- Date: (datatype not specified)
- CustId: (datatype not specified)
- Processing Rules / Functional Steps:
- 1. Connect to S3 location s3://adif-sdlc/sdlc_wizard/orderdata/

2. Read all CSV files present in the location
3. Parse CSV content with header row detection
4. Load data into a DataFrame structure preserving the schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Outputs:

- In-memory order dataset ready for data quality processing

- Preconditions:

- S3 bucket and path exist and are accessible
- CSV files contain header row matching the expected schema
- AWS Glue job has read permissions on the S3 location

- Postconditions:

- Order data is loaded into memory
- Record count is logged for audit purposes

- Error / Validation Conditions:

- If S3 path does not exist, log error and halt processing
- If CSV files are malformed or unreadable, log error and halt processing
- If schema does not match expected columns, log warning and attempt best-effort mapping

-

FR-DQ-001: Data Cleaning for Customer and Order Datasets

- Title: Remove Null Values and Duplicates

- Description:

The system shall perform data quality operations on both customer and order datasets by removing records containing null values (both string "Null" and actual NULL) and eliminating duplicate records.

- Business Objective:

Ensure data quality and integrity by removing incomplete and redundant records before downstream processing.

- Inputs:

- In-memory customer dataset from FR-INGEST-001
- In-memory order dataset from FR-INGEST-002

- Processing Rules / Functional Steps:

1. For customer dataset:

- Identify and remove rows where any column contains the string "Null" (case-sensitive)
- Identify and remove rows where any column contains actual NULL values
- Remove duplicate rows based on all columns
- Log count of records removed at each step

2. For order dataset:

- Identify and remove rows where any column contains the string "Null" (case-sensitive)
- Identify and remove rows where any column contains actual NULL values
- Remove duplicate rows based on all columns
- Log count of records removed at each step

- Outputs:

- Cleaned customer dataset
- Cleaned order dataset
- Data quality metrics (records removed, records retained)

- Preconditions:

- Customer and order datasets are loaded in memory

- Postconditions:

- Both datasets contain no null values or duplicates
- Data quality metrics are logged
- Error / Validation Conditions:
- If all records are removed due to null values, log critical warning
- If duplicate removal rate exceeds 50%, log warning for investigation

-

FR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Title: Create Glue Catalog Tables for Customer and Order Data

- Description:

The system shall register the cleaned customer and order datasets as tables in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Make cleaned datasets discoverable and queryable through AWS Athena and other AWS analytics services.

- Inputs:

- Cleaned customer dataset from FR-DQ-001
- Cleaned order dataset from FR-DQ-001
- Processing Rules / Functional Steps:
 1. Create or verify existence of Glue database: gen_ai_poc_databrickscoe
 2. Register customer dataset as table: sdlc_wizard_customer
- Infer or apply schema from customer dataset
- Set table properties and metadata
- 3. Register order dataset as table: sdlc_wizard_order
- Infer or apply schema from order dataset
- Set table properties and metadata
- 4. Update catalog with partition information if applicable
- Outputs:
 - Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer
 - Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_order
- Preconditions:
 - AWS Glue job has permissions to create/update Glue Catalog resources
 - Cleaned datasets are available
- Postconditions:
 - Tables are queryable via Athena
 - Table schemas are registered in Glue Catalog
- Error / Validation Conditions:
 - If database creation fails, log error and halt processing
 - If table registration fails, log error and halt processing
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with SCD Type 2 Logic Using Apache Hudi
- Description:

The system shall join customer and order data, implement Slowly Changing Dimension Type 2 logic to track historical changes, and store the result in Apache Hudi format on S3. When changes are detected, the system shall insert new active rows and close previous rows by updating their IsActive flag and EndDate.

- Business Objective:

Maintain complete historical tracking of order summaries to support time-based analysis and auditing requirements.

- Inputs:
 - Cleaned customer dataset from FR-DQ-001
 - Cleaned order dataset from FR-DQ-001
 - Existing ordersummary Hudi table (for incremental runs)
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. For initial load:
 - Create new records with IsActive = true
 - Set StartDate to current processing date
 - Set EndDate to null or a far-future date (e.g., 9999-12-31)
 - Set OpTs to current timestamp
 3. For incremental loads:
 - Compare incoming joined records with existing active records in ordersummary
 - For changed records:
 - Update existing active record: set IsActive = false, set EndDate to current processing date
 - Insert new record: set IsActive = true, set StartDate to current processing date, set EndDate to null
 - Set OpTs to current timestamp
 - For unchanged records: no action
 - For new records: insert with IsActive = true
 4. Write results to Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 5. Use Hudi's upsert operation for efficient SCD2 implementation
- Outputs:
 - Hudi table: ordersummary
 - Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
 - Schema includes: joined customer and order fields plus IsActive, StartDate, EndDate, OpTs
- Preconditions:
 - Cleaned customer and order datasets are available
 - S3 location for ordersummary is accessible
 - Hudi libraries are available in Glue environment
- Postconditions:

- All historical changes are tracked with proper SCD2 attributes
- Only one active record exists per unique business key at any point in time
- Closed records have IsActive = false and valid EndDate
- Error / Validation Conditions:
- If join results in no records, log warning
- If Hudi write operation fails, log error and halt processing
- If multiple active records exist for same business key, log critical error
-

FR-INCR-001: Incremental Processing Based on Customer Updates

- Title: Trigger SCD2 Updates When Customer Data Changes
- Description:

The system shall detect changes in customer data and propagate those changes to the ordersummary table using SCD Type 2 logic, ensuring that customer attribute changes are reflected in the historical order summary.

- Business Objective:

Maintain data consistency across related tables and ensure that customer profile changes are historically tracked in order summaries.

- Inputs:
- New/updated customer data from FR-INGEST-001
- Existing ordersummary Hudi table
- Processing Rules / Functional Steps:
- 1. Identify customer records that have changed since last processing run
- 2. Retrieve all order records associated with changed customers
- 3. Trigger SCD2 update logic (FR-SCD2-001) for affected order summary records
- 4. Ensure IsActive, StartDate, EndDate, and OpTs are properly maintained
- Outputs:
- Updated ordersummary Hudi table with new versions for affected records
- Preconditions:
- Customer change detection mechanism is in place
- Ordersummary table exists
- Postconditions:

- All order summaries related to changed customers have new active versions
- Previous versions are properly closed
- Error / Validation Conditions:
 - If change detection fails, log error and process all records as full refresh
 - If no changes detected, log informational message and skip processing
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table
- Description:

The system shall aggregate order data by customer to calculate total spending and store the results in a dedicated analytics table for business intelligence purposes.

- Business Objective:

Provide summarized customer spending metrics to support business analysis, reporting, and decision-making.

- Inputs:
 - Cleaned customer dataset from FR-DQ-001
 - Cleaned order dataset from FR-DQ-001
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. Calculate TotalAmount per customer: $\text{SUM}(\text{PricePerUnit} \times \text{Qty})$ grouped by customer
 3. Include customer Name and Date in the output
 4. Write aggregated results to S3 location: s3://adif-sdlc/analytics/customeraggregatespend/
 5. Register as Glue Catalog table: customeraggregatespend in database gen_ai_poc_databrickscoe
- Outputs:
 - Analytics table: customeraggregatespend
 - Location: s3://adif-sdlc/analytics/customeraggregatespend/
 - Glue Catalog table: gen_ai_poc_databrickscoe.customeraggregatespend
 - Schema:
 - Name: (datatype not specified)
 - TotalAmount: (datatype not specified)
 - Date: (datatype not specified)

- Preconditions:
- Cleaned customer and order datasets are available
- S3 location for analytics table is accessible
- Postconditions:
- Customer aggregate spend data is available for querying via Athena
- Table is registered in Glue Catalog
- Error / Validation Conditions:
- If aggregation results in no records, log warning
- If TotalAmount calculation produces negative values, log warning for investigation
- If write operation fails, log error and halt processing
-

3. Non-Functional Considerations

Monitoring and Observability

- All AWS Glue jobs shall log execution metrics to Amazon CloudWatch
- Job success/failure status shall be tracked
- Data quality metrics (records processed, records rejected) shall be logged

Data Governance (Optional)

- AWS Lake Formation may be used for fine-grained access control
- Data catalog resources may be governed through Lake Formation policies

Performance

- Processing shall leverage Apache Spark distributed computing capabilities
- Hudi format shall be used for efficient upsert operations in SCD2 implementation

Scalability

- Solution shall handle growing data volumes through S3 and Glue's elastic scaling
- Partitioning strategies may be applied to optimize query performance (not specified in BRD)
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	1. Source Input	Customer CSV ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/	

FR-INGEST-002	1. Source Input	Order CSV ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-DQ-001	3. Data Cleaning	Null removal and deduplication	
FR-CATALOG-001	4. Glue Catalog Tables	Registration of sdlc_wizard_customer and sdlc_wizard_order	
FR-SCD2-001	5. SCD Type 2	Hudi-based SCD2 implementation for ordersummary	
FR-INCR-001	6. Incremental Logic	Customer update-triggered SCD2 processing	
FR-AGG-001	7. Aggregation Table	Customer aggregate spend calculation and storage	

- Traceability to AWS Architecture Components (Section 8):
- Storage: S3 → All FR-INGEST, FR-SCD2, FR-AGG requirements
- ETL: AWS Glue (Spark + Hudi) → All processing requirements
- Catalog: Glue Data Catalog → FR-CATALOG-001, FR-SCD2-001, FR-AGG-001
- Query: Athena → Implicit in catalog registration requirements
- Monitoring: CloudWatch → Non-functional considerations
- Optional: Lake Formation → Non-functional considerations
-
- Document Version: 1.0
- Date: Generated from BRD (s3://adif-sdlc/brd_usecase1/BusinessRequirementDocument_usecase 1.pdf)